Individual Viva Defense & Code Implementation Master Scripts

Task-by-Task Defense Scripts for Pratik Sawant, Anvesh Patil, and Jayesh Deshmukh

Student Name & Roll No.	Assigned Implementation Tasks	Code Files Owned	Core Viva Defense Topics
Pratik Sawant 20240802324	• Task O2-T1: Multi-scale PyTorch layer hooks • Task O2-T2: 99% greedy coreset subsampling	patchcore.py (Core Algorithmic Engine)	Layer 2 & 3 rationale, 3x3 local pooling, Johnson-Lindenstrauss lemma, Min-Max Greedy k-Center math.
Anvesh Patil 20240802334	• Task O1-T1: MVTec dataset & mask pipeline • Task O4-T1: EfficientAD distillation framework	dataset.py & EfficientAD Benchmark	MVTec AD directory hierarchy, nearest-neighbor mask interpolation, ImageNet normalization, Student-Teacher distillation.
Jayesh Deshmukh 20240802335	• Task O3-T1: Nearest-neighbor scoring & smoothing • Task O5-T1: Streamlit UI & SQLite audit logging	app.py, db_logger.py, predict()	Euclidean distance scoring, 2D Gaussian filter (sigma=4), JET colormap alpha blending, SQLite ACID traceability.

1. Pratik Sawant (Tasks O2-T1 & O2-T2) — Algorithmic Engine

Pratik's 90-Second Opening Verbal Pitch:

"Good morning respected panel. In our project, my core responsibility was the algorithmic engine of PatchCore in patchcore.py. For task O2-T1, I implemented multi-scale patch feature extraction from scratch using PyTorch forward hooks. Rather than taking the final classification layer—which discards spatial information—I intercepted intermediate representations from layer2 and layer3 of ResNet-18, applying a 3x3 stride-1 average pooling layer to incorporate local spatial context into every patch. For task O2-T2, extracting patches across 209 normal training bottles generated 163,856 high-dimensional vectors (each 384-D). Storing all of them causes massive memory overhead and query latency. To solve this, I implemented dimensionality reduction using the Johnson-Lindenstrauss lemma down to 128 dimensions, followed by the Min-Max Greedy k-Center coreset algorithm. This compressed our memory bank by 99%—from 163,856 down to just 1,638 representative vectors—with virtually zero loss in detection accuracy."

Pratik's Code Walkthrough Lines to Point to:

• **PyTorch Forward Hooks:** self.backbone.layer2.register_forward_hook(...) intercepts intermediate activations without breaking forward inference.
• **Spatial Aggregation:** avg_pool = nn.AvgPool2d(3, stride=1, padding=1) pools 3x3 patch neighbors for vibration invariance; bilinearly upsamples layer3 to match layer2, concatenating into 384-D patch tokens.
• **Greedy k-Center:** _k_center_greedy() iteratively selects the sample whose minimum distance to existing centers is maximum (np.argmax(dist_to_centers)).

Pratik's Key Viva Defense Q&A;:

Q: Why didn't you use layer4?
Answer: Layer 4 has 32x downsampling (224x224 becomes 7x7). The receptive field is too broad, destroying spatial localization for 2mm micro-cracks. Layer 4 is also over-specialized to ImageNet classification categories. Layers 2 and 3 strike the optimal Goldilocks balance.

Q: What is the Johnson-Lindenstrauss guarantee?
Answer: It mathematically proves that points in high dimensions can be linearly projected into a lower dimension while preserving pairwise Euclidean distances within (1 +/- epsilon). Projecting from 384-D to 128-D cuts distance operations by 66% with negligible metric distortion.

2. Anvesh Patil (Tasks O1-T1 & O4-T1) — Data & Distillation

Anvesh's 90-Second Opening Verbal Pitch:

"Good morning respected faculty. My primary responsibilities covered dataset architecture engineering in dataset.py and the EfficientAD distillation comparison framework for task O4-T1. For task O1-T1, the standard torchvision ImageFolder class cannot handle MVTec AD because the directory structure is non-standard: test images are grouped by defect types, while ground-truth pixel masks are stored in a parallel directory tree. I wrote dataset.py from scratch to automatically pair every defective test image with its corresponding 1-channel binary mask, while applying ImageNet normalization and nearest-neighbor mask resizing. For task O4-T1, I analyzed and benchmarked EfficientAD. While PatchCore is a memory-bank method with zero training epochs, its query latency is high. EfficientAD solves this by using a pre-trained Teacher Network and training a Student Network to mimic the Teacher on normal images. I configured the 5-class evaluation pipeline (bottle, cable, capsule, metal_nut, transistor) to measure the trade-off between PatchCore's memory lookups and EfficientAD's sub-10ms forward passes."

Anvesh's Code Walkthrough Lines to Point to:

• **Ground Truth Mask Pairing:** mask_path = os.path.join(root, 'ground_truth', defect, f'{name}_mask.png') maps defective test images to 1-channel binary segmentation ground-truth masks.
• **Nearest-Neighbor Mask Transform:** Resize((224, 224), interpolation=InterpolationMode.NEAREST) preserves strict integer 0/1 binary values without bilinear edge blur.
• **ImageNet Normalization:** mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225] ensures input tensors match the channel distribution of pre-trained convolutional filters.

Anvesh's Key Viva Defense Q&A;:

Q: How does EfficientAD detect anomalies without a memory bank?
Answer: Through knowledge distillation. The Teacher is pre-trained and frozen. The Student is trained strictly on normal images to predict the Teacher's output. When normal parts pass, Student matches Teacher (difference ~ 0). When a crack appears, the Student fails to mimic the Teacher (high difference flags defect in a single forward pass).

Q: Why train EfficientAD on only 5 classes instead of all 15?
Answer: PatchCore has 0 training epochs (~15s per class). EfficientAD requires ~70,000 backprop iterations per class (over 1M iterations total). Benchmarking 5 representative object and texture classes provides complete empirical trade-off data within our 12-week compute budget.

3. Jayesh Deshmukh (Tasks O3-T1 & O5-T1) — Scoring & Deployment

Jayesh's 90-Second Opening Verbal Pitch:

"Good morning respected panel. My focus in this project was the inference scoring pipeline for task O3-T1 and the complete deployment architecture for task O5-T1. For task O3-T1, I implemented the nearest-neighbor distance computation in patchcore.py. For every query patch in a test image, my code computes the Euclidean distance to the nearest patch in Pratik's memory bank. I then interpolate this distance grid back to the original 224x224 resolution and apply a 2D Gaussian filter (sigma = 4) using SciPy. This eliminates high-frequency noise spikes and produces smooth, contiguous anomaly heatmaps. On our test run, defective bottles scored 3.28 while normal bottles scored 1.30, providing a clean 1.98 decision margin. For task O5-T1, I built our interactive Streamlit application (app.py) and integrated an ACID-compliant SQLite audit trail (db_logger.py) following GE Aerospace exception logging standards. This allows factory operators to inspect parts, view live JET heatmap overlays, adjust decision thresholds, and permanently record inspection events for ISO 9001 quality audits."

Jayesh's Code Walkthrough Lines to Point to:

• **Nearest-Neighbor Distance:** dists = torch.norm(patches.unsqueeze(1) - memory_bank.unsqueeze(0), p=2, dim=2) computes minimum Euclidean distances to the normal coreset.
• **2D Gaussian Smoothing:** gaussian_filter(anomaly_map, sigma=4) smooths coarse 28x28 distance spikes into continuous defect regions; image_score = float(np.max(anomaly_map)) extracts image verdict.
• **SQLite Audit Logger:** db_logger.py inserts timestamp, part category, model, raw score, threshold, verdict, latency, and image name into inspections.db.
• **Streamlit JET Alpha Blending:** overlay = np.uint8(0.6 * img_np + 0.4 * heatmap_color) renders the operator visualization in app.py.

Jayesh's Key Viva Defense Q&A;:

Q: Why apply a Gaussian filter?
Answer: Raw nearest-neighbor patch scores exist on a coarse 28x28 grid. Bilinear upsampling alone leaves blocky artifacts and single-pixel false alarms caused by sensor noise. A 2D Gaussian filter (sigma = 4) smooths noise spikes into physical defect contours.

Q: Why use SQLite instead of a flat CSV or MongoDB?
Answer: In manufacturing lines, multiple inspection threads write simultaneously. A flat CSV lacks ACID transactions and corrupts during concurrent writes. MongoDB introduces network latency and server overhead. SQLite is serverless, zero-configuration, ACID-compliant, and embeds directly into industrial edge inspection terminals.